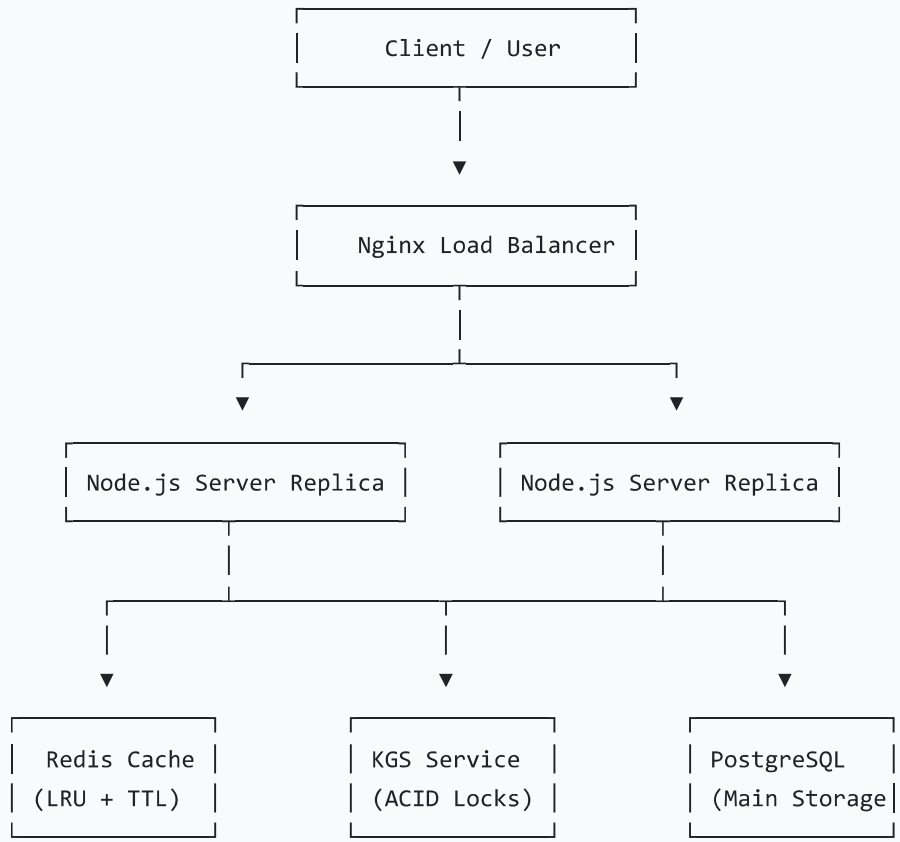
# 📚 URL Shortener (TinyURL) — System Design Guide & Architecture Document

## 📊 Overall Project Status

| Component | Status | Description |
| --- | --- | --- |
| **Backend API & Core Logic** | **100% Complete** | Express.js, KGS, Cache-Aside, Rate Limiter, Async Analytics |
| **PostgreSQL Database & Schema** | **100% Complete** | Pre-allocated `available_keys`, `urls`, `url_clicks` schema |
| **Redis Caching & Rate Limiting** | **100% Complete** | Cache-Aside pattern + Token Bucket DDoS protection |
| **Nginx Load Balancer** | *Next Step (Step 6)* | Distributing traffic across multiple API container replicas |
| **React + Tailwind Frontend UI** | *Next Step (Step 7)* | Interactive Shortener Dashboard & Live System Telemetry UI |

## 📐 1. High-Level Architecture Diagram




## 💡 2. Key System Design Concepts Implemented

### 1️⃣ Data Estimation & Capacity Planning (Math)

- **Target Volume:** $1,000$ writes/sec, $10,000 - 100,000$ reads/sec ($10:1$ to $100:1$ read-heavy).
- **10-Year Volume:** $1,000 \text{ writes/sec} \times 315.36 \text{ million sec/year} \times 10 \text{ years} \approx 315 \text{ Billion URLs}$
  1,000 writes/sec × 315.36 million sec/year × 10 years ≈ 315 Billion URLs.
- **Character Set & Key Length:** Base62 $[0-9, a-z, A-Z]$[0 – 9, a – z, A – Z] (62 characters).
  $\text{Capacity} = 62^7 = 3,521,614,606,208 \approx 3.5 \text{ Trillion URLs}$
  Capacity = $627 = 3,521,614,606,208 \approx 3.5$ Trillion URLs A 7-character string easily covers 10+ years of operation with zero truncation risk.

## 2️⃣ Key Generation Service (KGS) & Zero-Collision Guarantee

- **The Problem:** Dynamic hashing (`MD5` or `CRC32`) produces hash collisions when different long URLs map to the same short key.
- **The Solution:** We pre-generate 7-character Base62 keys in the `available_keys` SQL table.
- **ACID Row-Level Lock:** When multiple web servers request a new key simultaneously, PostgreSQL executes:

```sql
SELECT key FROM available_keys
WHERE is_used = FALSE
LIMIT 1
FOR UPDATE SKIP LOCKED;
```

  - `FOR UPDATE`: Locks the row so no other server can select it.
  - `SKIP LOCKED`: Tells concurrent transactions to immediately grab the *next* free row instead of waiting, eliminating database lock contention.

## 3️⃣ Redis Cache-Aside Pattern (Low Latency Reads)

- **Goal:** Sub-millisecond read latency for high-frequency short link redirects.
- **Flow:**
  1. Client hits `GET /:shortCode`.
  2. Server queries Redis first (`url:<shortCode>`).
  3. **CACHE HIT (~1ms~ 1ms):** Returns cached JSON payload immediately.
  4. **CACHE MISS (~20ms~ 20ms):** Queries PostgreSQL, caches the payload in Redis with `EX` (TTL) policy, and then returns.

## 4️⃣ HTTP 302 Found vs 301 Moved Permanently

- **HTTP 301 (Permanent Redirect):** Browser caches the redirect locally. Subsequent clicks bypass the backend entirely.
- **HTTP 302 (Temporary Redirect):** Every click passes through our server. **We chose 302** because it allows us to record click analytics (IP, Referrer, User-Agent, Timestamp) accurately for every request.

### 5️⃣ Non-Blocking Asynchronous Click Tracking

- **Problem:** Inserting analytics logs into SQL synchronously increases redirect response time by $20 - 50$ ms$20 - 50$ ms.
- **Solution:** The redirect response ( `res.redirect(302, longUrl)` ) is returned immediately. The click counter increment and `url_clicks` log insertion are executed asynchronously in the background.

### 6️⃣ Redis Sliding Window Rate Limiter

- **DDoS Protection:** Uses Redis atomic `INCR` and `EXPIRE` commands tied to the client's IP address ( `ratelimit:<ip>` ).
- If a single IP exceeds $100$ req/min100 req/min, the server returns **HTTP 429 Too Many Requests**.

## 🎯 3. Top Interview Questions & How to Answer

### Q1: "How do you handle hash collisions?"

> *"Instead of generating short keys dynamically via hashing on write requests, we use a dedicated **Key Generation Service (KGS)** that pre-allocates unique 7-character Base62 keys in PostgreSQL. We enforce zero race conditions using PostgreSQL row-level locks with `FOR UPDATE SKIP LOCKED`."*

### Q2: "How does your system handle 100,000 reads per second?"

> *"Our system uses a **Redis Cache-Aside** strategy. Popular URLs are cached in-memory with LRU and TTL eviction policies. Redis handles $90\% + 90\%+$ of read traffic with sub-millisecond latency, preventing our PostgreSQL database from being bottlenecked."*

### Q3: "Why did you use HTTP 302 instead of HTTP 301?"

> *"HTTP 301 forces browsers to cache redirects indefinitely, which prevents us from gathering real-time telemetry. HTTP 302 ensures every request reaches our server so we can track click counts, geographical referrers, and user-agent data asynchronously."*

## 🚀 4. API Endpoints Quick Reference

| Method | Endpoint | Description | Status Code |
| --- | --- | --- | --- |
| `POST` | `/api/urls` | Shorten a long URL | `201 Created` |
| `GET` | `/:shortCode` | Redirect to original long URL | `302 Found` |
| `GET` | `/api/urls/:shortCode/analytics` | Fetch click statistics & referrers | `200 OK` |
| `GET` | `/health` | Check PostgreSQL and Redis status | `200 OK` |